

UChat-Style Video Generation Platform — Application Architecture

1. Overview

A multi-tenant SaaS platform for AI-powered video generation (script → voice → video) with workspace-based hierarchy. Each workspace is an isolated environment for a company/business, supporting role-based access control (RBAC) and third-party API integrations.

Tech Stack: Next.js (Frontend) + Supabase (PostgreSQL + Auth) + Third-party APIs (Voice, Video, Publishing)

2. Hierarchy Model

```
Platform Owner (Super Admin)
|
├─ Workspace A (Client A / Company A)
|   │   └─ Admin (Organization Manager)
|   │       │   └─ Users (Video Creators)
|   │       │   └─ Editors (Video Editors)
|   │       └─ Support Agents / Viewers
|   └─ Projects (within Workspace A)
|       │   └─ Project 1 → Script → Voice → Video → Edit → Publish
|       └─ Project 2 → Script → Voice → Video → Edit → Publish
|
├─ Workspace B (Client B / Company B)
|   │   └─ Admin
|   │       │   └─ Users
|   │       └─ Editors
|   └─ Projects
|
└─ Workspace C (Client C)
    └─ ...
```

Key Principle: One Owner can manage multiple Workspaces. Each Workspace is fully isolated. Users can belong to multiple workspaces with different roles.

3. Database Schema (PostgreSQL + Supabase)

3.1 Core Tables

```
-- =====
-- TENANTS & WORKSPACES
-- =====

CREATE TABLE workspaces (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
```

```

slug TEXT UNIQUE NOT NULL,          -- URL-friendly identifier
owner_id UUID REFERENCES auth.users(id), -- Platform Owner
description TEXT,
logo_url TEXT,
status TEXT DEFAULT 'active',        -- active, suspended, archived
subscription_tier TEXT DEFAULT 'basic',
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now(),
metadata JSONB DEFAULT '{}'          -- Custom config per workspace
);

-- =====
-- USERS (Global Identity)
-- =====

CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT NOT NULL UNIQUE,
  full_name TEXT,
  avatar_url TEXT,
  phone TEXT,
  is_platform_owner BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- WORKSPACE MEMBERSHIPS (RBAC)
-- =====

CREATE TABLE workspace_members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  role TEXT NOT NULL CHECK (role IN ('owner', 'admin', 'user', 'editor', 'support', 'viewer')),
  status TEXT DEFAULT 'active',        -- active, pending, suspended
  invited_by UUID REFERENCES profiles(id),
  temp_password TEXT,                  -- For initial account creation
  password_changed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(workspace_id, user_id)
);

-- =====
-- API INTEGRATIONS (Per Workspace)
-- =====

CREATE TABLE workspace_apis (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  provider TEXT NOT NULL,              -- 'voice_ai', 'video_ai', 'subagic', 'zernio', 'custom'
  api_name TEXT NOT NULL,              -- Display name
  api_key_encrypted TEXT NOT NULL,     -- Encrypted API key

```

```

    api_secret_encrypted TEXT,          -- Encrypted secret if applicable
    endpoint_url TEXT,
    is_active BOOLEAN DEFAULT TRUE,
    config JSONB DEFAULT '{}',          -- Provider-specific settings
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- PROJECTS
-- =====

CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    title TEXT NOT NULL,
    description TEXT,
    status TEXT DEFAULT 'idea',          -- idea, scripting, voice_gen, video_gen, editing, review,
    -- published, archived
    priority TEXT DEFAULT 'medium',      -- low, medium, high, urgent
    created_by UUID REFERENCES profiles(id),
    assigned_to UUID REFERENCES profiles(id), -- Primary user responsible
    deadline TIMESTAMPTZ,
    progress_percent INTEGER DEFAULT 0 CHECK (progress_percent BETWEEN 0 AND 100),
    thumbnail_url TEXT,
    video_url TEXT,
    published_urls JSONB DEFAULT '[]', -- Array of {platform, url, published_at}
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- PROJECT TIMELINE / STAGES
-- =====

CREATE TABLE project_stages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    stage_name TEXT NOT NULL,            -- idea, script, voice, video, editing, review, publish
    status TEXT DEFAULT 'pending',       -- pending, in_progress, completed, failed, skipped
    started_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    assigned_to UUID REFERENCES profiles(id),
    notes TEXT,
    output_data JSONB DEFAULT '{}',      -- Stage-specific output (script text, voice URL, etc.)
    created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- SCRIPTS
-- =====

CREATE TABLE scripts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
content TEXT NOT NULL,           -- The actual script text
tone TEXT DEFAULT 'professional', -- professional, casual, energetic, etc.
language TEXT DEFAULT 'en',
word_count INTEGER,
estimated_duration INTEGER,      -- In seconds
ai_generated BOOLEAN DEFAULT FALSE,
created_by UUID REFERENCES profiles(id),
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- VOICE GENERATIONS
-- =====

CREATE TABLE voice_generations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  script_id UUID REFERENCES scripts(id),
  voice_provider TEXT NOT NULL,    -- e.g., 'elevenlabs', 'azure_tts', etc.
  voice_id TEXT NOT NULL,         -- Specific voice model ID
  audio_url TEXT,
  duration INTEGER,              -- In seconds
  status TEXT DEFAULT 'pending',  -- pending, generating, completed, failed
  settings JSONB DEFAULT '{}',    -- Speed, pitch, emotion settings
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- VIDEO GENERATIONS
-- =====

CREATE TABLE video_generations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  voice_generation_id UUID REFERENCES voice_generations(id),
  video_provider TEXT NOT NULL,    -- e.g., 'heygen', 'synthesia', 'd-id'
  video_url TEXT,
  status TEXT DEFAULT 'pending',
  settings JSONB DEFAULT '{}',    -- Resolution, avatar, background, etc.
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- EDITING TASKS
-- =====

CREATE TABLE editing_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  editor_id UUID REFERENCES profiles(id),

```

```

requested_by UUID REFERENCES profiles(id), -- User who sent for editing
edit_type TEXT NOT NULL,                -- 'ai_subagic', 'manual_editor'
status TEXT DEFAULT 'pending',          -- pending, in_progress, review, completed, rejected
instructions TEXT,
source_video_url TEXT,
edited_video_url TEXT,
feedback TEXT,
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now(),
completed_at TIMESTAMPTZ
);

-- =====
-- PUBLISHING
-- =====
CREATE TABLE publishing_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  platform TEXT NOT NULL,                -- 'youtube', 'tiktok', 'instagram', 'linkedin', 'facebook',
  -- 'twitter'
  status TEXT DEFAULT 'draft',           -- draft, scheduled, publishing, published, failed
  scheduled_at TIMESTAMPTZ,
  published_at TIMESTAMPTZ,
  platform_video_id TEXT,                -- ID returned by platform API
  title TEXT,
  description TEXT,
  tags TEXT[],
  thumbnail_url TEXT,
  custom_thumbnail_url TEXT,             -- User-uploaded thumbnail
  visibility TEXT DEFAULT 'public',      -- public, unlisted, private
  settings JSONB DEFAULT '{}',           -- Platform-specific settings
  created_by UUID REFERENCES profiles(id),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- SCHEDULED TASKS / CALENDAR
-- =====
CREATE TABLE scheduled_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  task_type TEXT NOT NULL,               -- 'video_deadline', 'meeting', 'review', 'general'
  assigned_to UUID REFERENCES profiles(id), -- Can be user or editor
  assigned_by UUID REFERENCES profiles(id), -- Admin who assigned
  start_date TIMESTAMPTZ NOT NULL,
  end_date TIMESTAMPTZ,
  status TEXT DEFAULT 'pending',         -- pending, in_progress, completed, cancelled
  related_project_id UUID REFERENCES projects(id),

```

```

        priority TEXT DEFAULT 'medium',
        created_at TIMESTAMPTZ DEFAULT now(),
        updated_at TIMESTAMPTZ DEFAULT now()
    );

-- =====
-- NOTIFICATIONS
-- =====

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    workspace_id UUID REFERENCES workspaces(id),
    type TEXT NOT NULL,          -- 'deadline', 'edit_request', 'edit_complete',
    -- 'publish_complete', 'mention', 'system'
    title TEXT NOT NULL,
    message TEXT,
    related_project_id UUID REFERENCES projects(id),
    related_task_id UUID REFERENCES editing_tasks(id),
    is_read BOOLEAN DEFAULT FALSE,
    action_url TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- ACTIVITY LOGS (Audit Trail)
-- =====

CREATE TABLE activity_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    user_id UUID REFERENCES profiles(id),
    action TEXT NOT NULL,        -- 'project_created', 'video_generated', 'published', etc.
    entity_type TEXT NOT NULL,   -- 'project', 'video', 'user', 'workspace'
    entity_id UUID,
    details JSONB DEFAULT '{}',
    ip_address INET,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- ROW LEVEL SECURITY POLICIES
-- =====

-- Enable RLS on all tables
ALTER TABLE workspaces ENABLE ROW LEVEL SECURITY;
ALTER TABLE workspace_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE project_stages ENABLE ROW LEVEL SECURITY;
ALTER TABLE scripts ENABLE ROW LEVEL SECURITY;
ALTER TABLE voice_generations ENABLE ROW LEVEL SECURITY;
ALTER TABLE video_generations ENABLE ROW LEVEL SECURITY;
ALTER TABLE editing_tasks ENABLE ROW LEVEL SECURITY;

```

```

ALTER TABLE publishing_tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE scheduled_tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE notifications ENABLE ROW LEVEL SECURITY;
ALTER TABLE activity_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE workspace_apis ENABLE ROW LEVEL SECURITY;

-- Workspace Access Policy (Users can only see workspaces they belong to)
CREATE POLICY workspace_access ON workspaces
  FOR ALL USING (
    id IN (
      SELECT workspace_id FROM workspace_members
      WHERE user_id = auth.uid()
    ) OR owner_id = auth.uid()
  );

-- Project Access Policy (Projects scoped to workspace membership)
CREATE POLICY project_access ON projects
  FOR ALL USING (
    workspace_id IN (
      SELECT workspace_id FROM workspace_members
      WHERE user_id = auth.uid()
    )
  );

-- Similar policies apply to all tenant-scoped tables...

```

4. Frontend Application Structure (Next.js App Router)

```

app/
|
├─ (marketing)/                                # Public pages (no auth required)
|   ├─ page.tsx                                # Landing Page
|   ├─ layout.tsx
|   ├─ features/
|   |   └─ page.tsx                            # Features showcase
|   ├─ pricing/
|   |   └─ page.tsx
|   ├─ about/
|   |   └─ page.tsx
|   └─ contact/
|       └─ page.tsx
|
├─ (auth)/                                    # Authentication pages
|   ├─ layout.tsx
|   ├─ login/
|   |   └─ page.tsx
|   ├─ register/
|   |   └─ page.tsx
|   └─ forgot-password/

```

```

|   |   └─ page.tsx
|   └─ reset-password/
|       └─ page.tsx
|
|─ (dashboard)/                                # Protected dashboard
|   └─ layout.tsx                            # Dashboard shell with sidebar, nav
|   └─ page.tsx                              # Redirect to appropriate dashboard based on role
|
|   └─ owner/                                # OWNER DASHBOARD
|       └─ page.tsx                          # Owner overview / workspace list
|       └─ workspaces/
|           └─ page.tsx                      # List all workspaces
|           └─ new/
|               └─ page.tsx                  # Create new workspace
|               └─ [workspaceId]/
|                   └─ page.tsx              # Workspace detail
|                   └─ settings/
|                       └─ page.tsx          # Workspace settings
|                   └─ apis/
|                       └─ page.tsx          # List configured APIs
|                       └─ new/
|                           └─ page.tsx      # Add new API integration
|                   └─ members/
|                       └─ page.tsx          # Manage workspace members
|                       └─ invite/
|                           └─ page.tsx      # Invite new members
|                   └─ projects/
|                       └─ page.tsx          # Inspect all projects in workspace
|   └─ projects/
|       └─ page.tsx                          # Cross-workspace project inspection
|
|   └─ admin/                                # ADMIN DASHBOARD
|       └─ page.tsx                          # Admin overview
|       └─ projects/
|           └─ page.tsx                      # All team projects
|           └─ [projectId]/
|               └─ page.tsx                  # Project detail
|       └─ team/
|           └─ page.tsx                      # Team members list
|           └─ users/
|               └─ page.tsx                  # Manage users
|               └─ editors/
|                   └─ page.tsx              # Manage editors
|       └─ calendar/
|           └─ page.tsx                      # Scheduling / Calendar view
|                                           # Create deadlines, assign tasks
|       └─ workspace-settings/
|           └─ page.tsx                      # Admin workspace configuration
|
|   └─ user/                                # USER / CREATOR DASHBOARD
|       └─ page.tsx                          # User overview / My projects

```

```

| | | └─ projects/
| | | | └─ page.tsx # All my projects list
| | | | └─ new/
| | | | | └─ page.tsx # Create new project
| | | | | └─ [projectId]/
| | | | | | └─ page.tsx # Project detail & timeline
| | | | | | └─ script/
| | | | | | | └─ page.tsx # Script generation / editing
| | | | | | └─ voice/
| | | | | | | └─ page.tsx # Voice generation
| | | | | | └─ video/
| | | | | | | └─ page.tsx # Video generation
| | | | | | └─ edit/
| | | | | | | └─ page.tsx # Send for editing (AI or Editor)
| | | | | | └─ publish/
| | | | | | | └─ page.tsx # Publishing configuration
| | | └─ publish/
| | | | └─ page.tsx # Publishing hub
| | | | └─ upload/
| | | | | └─ page.tsx # Upload own video + thumbnail
| | | | | └─ [platform]/
| | | | | | └─ page.tsx # Platform-specific publish settings
| | | └─ notifications/
| | | | └─ page.tsx # Deadlines, alerts, mentions
| | └─ editor/ # EDITOR DASHBOARD
| | | └─ page.tsx # Editor overview / Assigned tasks
| | | └─ tasks/
| | | | └─ page.tsx # All editing tasks
| | | | | └─ [taskId]/
| | | | | | └─ page.tsx # Task detail
| | | | | | └─ edit/
| | | | | | | └─ page.tsx # AI editing interface (Subagic)
| | | └─ notifications/
| | | | └─ page.tsx # Deadlines & requests
| | | └─ completed/
| | | | └─ page.tsx # Completed edits history
| └─ api/ # API Routes (Next.js)
| | └─ auth/
| | | └─ [...nextauth]/
| | | | └─ route.ts # Supabase Auth integration
| | └─ webhooks/
| | | └─ voice/
| | | | └─ route.ts # Voice generation webhook
| | | └─ video/
| | | | └─ route.ts # Video generation webhook
| | | └─ publish/
| | | | └─ route.ts # Publishing webhook
| | └─ integrations/
| | | └─ voice/

```

```

|   |   └─ route.ts          # Voice API proxy
|   └─ video/
|       └─ route.ts          # Video API proxy
|       └─ publish/
|           └─ route.ts      # Zernio API proxy
|
└─ layout.tsx                # Root layout
└─ globals.css
└─ error.tsx

components/
|
└─ ui/                        # Reusable UI components (shadcn/ui)
    └─ button.tsx
    └─ card.tsx
    └─ dialog.tsx
    └─ dropdown-menu.tsx
    └─ table.tsx
    └─ tabs.tsx
    └─ calendar.tsx
    └─ timeline.tsx
    └─ progress.tsx
    └─ ...

└─ layout/                    # Layout components
    └─ dashboard-shell.tsx    # Main dashboard wrapper
    └─ sidebar.tsx            # Navigation sidebar
    └─ topbar.tsx             # Top navigation bar
    └─ workspace-switcher.tsx # Workspace dropdown switcher
    └─ mobile-nav.tsx

└─ auth/                      # Auth-specific components
    └─ login-form.tsx
    └─ register-form.tsx
    └─ password-reset-form.tsx
    └─ role-guard.tsx         # Role-based route protection

└─ projects/                  # Project-related components
    └─ project-card.tsx
    └─ project-timeline.tsx   # Visual timeline (Idea → Script → Voice → Video → Edit →
    ↪ Publish)
    └─ project-create-modal.tsx
    └─ stage-badge.tsx
    └─ progress-tracker.tsx

└─ video/                     # Video generation components
    └─ script-editor.tsx
    └─ voice-selector.tsx
    └─ voice-player.tsx
    └─ video-preview.tsx
    └─ video-settings-panel.tsx

```

```

|   └─ publish-platform-selector.tsx
|
└─ editing/                                # Video editing components
    │   └─ ai-edit-panel.tsx                # Subagic integration UI
    │   └─ manual-edit-request.tsx
    │   └─ video-comparison.tsx            # Before/after view
    │   └─ edit-feedback-form.tsx
    |
└─ publishing/                             # Publishing components
    │   └─ platform-cards.tsx               # YouTube, TikTok, etc.
    │   └─ publish-form.tsx
    │   └─ thumbnail-uploader.tsx
    │   └─ video-uploader.tsx
    │   └─ schedule-picker.tsx
    │   └─ platform-preview.tsx
    |
└─ calendar/                              # Calendar components
    │   └─ task-calendar.tsx
    │   └─ deadline-card.tsx
    │   └─ task-assign-modal.tsx
    |
└─ workspace/                             # Workspace management
    │   └─ workspace-card.tsx
    │   └─ member-list.tsx
    │   └─ invite-member-modal.tsx
    │   └─ api-config-form.tsx
    │   └─ role-badge.tsx
    |
└─ notifications/                         # Notification system
    │   └─ notification-bell.tsx
    │   └─ notification-list.tsx
    │   └─ notification-item.tsx
    |
hooks/
└─ use-auth.ts                            # Authentication & role checking
└─ use-workspace.ts                       # Current workspace context
└─ use-projects.ts                        # Project CRUD operations
└─ use-notifications.ts                   # Real-time notifications
└─ use-api-integration.ts                 # Third-party API management
└─ use-video-generation.ts                # Video generation workflow
└─ use-role-guard.ts                      # Permission checking

lib/
└─ supabase/
    │   └─ client.ts                       # Browser client
    │   └─ server.ts                       # Server-side client
    │   └─ admin.ts                        # Service role client
    |
└─ api/
    │   └─ voice-api.ts                    # Voice generation service
    │   └─ video-api.ts                   # Video generation service
    │   └─ subagic-api.ts                 # AI editing service

```

```

|   └─ zernio-api.ts           # Social publishing service
└─ auth/
|   └─ roles.ts               # Role definitions & permissions
|   └─ permissions.ts         # Permission matrix
└─ utils/
|   └─ cn.ts                  # Class name utility
|   └─ date.ts                # Date formatting
|   └─ validators.ts          # Form validation schemas
└─ constants.ts               # App constants

types/
└─ index.ts                   # Common types
└─ auth.ts                    # Auth-related types
└─ workspace.ts               # Workspace types
└─ project.ts                 # Project & workflow types
└─ video.ts                   # Video generation types
└─ api.ts                     # API integration types
└─ notification.ts            # Notification types

middleware.ts                  # Route protection & role redirects
next.config.js
tailwind.config.ts
package.json

```

5. Role-Based Access Control (RBAC)

5.1 Permission Matrix

Feature	Owner	Admin	User	Editor	Support	Viewer
Workspace Management						
Create Workspace	✓	✗	✗	✗	✗	✗
Delete Workspace	✓	✗	✗	✗	✗	✗
Configure APIs	✓	✗	✗	✗	✗	✗
View Workspace	✓	✓	✓	✓	✓	✓
Member Management						
Invite Members	✓	✓	✗	✗	✗	✗
Remove Members	✓	✓	✗	✗	✗	✗

Table 1 – continued

Feature	Owner	Admin	User	Editor	Support	Viewer
Change Roles	✓	✓	✗	✗	✗	✗
Reset Passwords	✓	✓	✗	✗	✗	✗
Projects						
Create Project	✓	✓	✓	✗	✗	✗
View All Projects	✓	✓	✓*	✓*	✗	✗
Edit Any Project	✓	✓	✗	✗	✗	✗
Delete Project	✓	✓	✓ (own)	✗	✗	✗
Video						
Workflow						
Generate Script	✓	✓	✓	✗	✗	✗
Generate Voice	✓	✓	✓	✗	✗	✗
Generate Video	✓	✓	✓	✗	✗	✗
Send for Editing	✓	✓	✓	✗	✗	✗
AI Edit (Subagic)	✓	✓	✗	✓	✗	✗
Manual Edit	✓	✓	✗	✓	✗	✗
Publishing						
Publish to Social	✓	✓	✓	✗	✗	✗
Schedule Posts	✓	✓	✓	✗	✗	✗
Upload Custom Video	✓	✓	✓	✗	✗	✗
Calendar / Scheduling						
Create Deadlines	✓	✓	✗	✗	✗	✗
Assign Tasks	✓	✓	✗	✗	✗	✗
View Calendar	✓	✓	✓	✓	✓	✓

Table 1 – continued

Feature	Owner	Admin	User	Editor	Support	Viewer
Administration						
View	✓	✓	✗	✗	✗	✗
Activity						
Logs						
View	✓	✓	✗	✗	✗	✗
Analytics						
Platform	✓	✗	✗	✗	✗	✗
Settings						

*Users see their own projects; Editors see projects assigned to them.

5.2 Middleware Implementation

```
// middleware.ts
import { createMiddlewareClient } from '@supabase/auth-helpers-nextjs';
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

const roleRoutes = {
  '/dashboard/owner': ['owner'],
  '/dashboard/admin': ['admin', 'owner'],
  '/dashboard/user': ['user', 'admin', 'owner'],
  '/dashboard/editor': ['editor', 'admin', 'owner'],
};

export async function middleware(req: NextRequest) {
  const res = NextResponse.next();
  const supabase = createMiddlewareClient({ req, res });

  const { data: { session } } = await supabase.auth.getSession();

  if (!session) {
    return NextResponse.redirect(new URL('/login', req.url));
  }

  // Check role-based access
  const path = req.nextUrl.pathname;
  const { data: member } = await supabase
    .from('workspace_members')
    .select('role')
    .eq('user_id', session.user.id)
    .single();

  const userRole = member?.role || 'viewer';

  // Redirect based on role if hitting root dashboard
  if (path === '/dashboard') {
```

```

const redirectPath = getDefaultDashboard(userRole);
return NextResponse.redirect(new URL(redirectPath, req.url));
}

return res;
}

```

6. Key Workflows

6.1 New Project Creation (User Flow)

User Dashboard → New Project

- |
- ├ Enter Project Title & Description
- ├ Select AI Script Generation OR Write Manual Script
 - | ─ AI: Enter topic, tone, duration → Generate Script
 - | ─ Manual: Write script in editor
- |
- ├ Voice Generation Stage
 - | ─ Select Voice Provider (from workspace APIs)
 - | ─ Choose Voice Model
 - | ─ Configure Settings (speed, pitch, emotion)
 - | ─ Generate → Preview Audio
- |
- ├ Video Generation Stage
 - | ─ Select Video Provider (from workspace APIs)
 - | ─ Configure Avatar/Scene/Background
 - | ─ Upload assets if needed
 - | ─ Generate → Preview Video
- |
- ├ Editing Stage
 - | ─ Option A: Send to AI Editor (Subagic)
 - | ─ Configure edit instructions → AI processes
 - | ─ Option B: Send to Human Editor
 - | ─ Select editor → Add instructions → Submit
 - | ─ Option C: Skip editing
- |
- └ Publishing Stage
 - ├ Upload Custom Thumbnail (optional)
 - ├ Select Platforms (YouTube, TikTok, Instagram, etc.)
 - ├ Write Title, Description, Tags per platform
 - ├ Schedule or Publish Immediately
 - └ Confirm → Publish via Zernio API

6.2 Admin Scheduling Flow

Admin Dashboard → Calendar

- |
- ├ Click Date → Create Task

- └─ Select Task Type: Video Deadline
- └─ Select User/Editor to assign
- └─ Select Related Project (optional)
- └─ Set Start & End Date/Time
- └─ Add Instructions/Description
- └─ Set Priority
- └─ Save → Notification sent to assignee

6.3 Editor Workflow

```
Editor Dashboard
|
└─ View Incoming Edit Requests (from Users)
└─ Accept / Decline Task
└─ AI Edit Path:
|   └─ Open Subagic Integration
|   └─ Apply AI edits based on instructions
|   └─ Review output
|   └─ Send to User for Review
└─ Manual Edit Path:
|   └─ Download source video
|   └─ Edit in external tool (or built-in editor)
|   └─ Upload edited version
|   └─ Send to User for Review
└─ User approves → Video moves to Publish stage
```

6.4 Owner Workspace Inspection

```
Owner Dashboard → Workspaces
|
└─ List all owned workspaces
└─ Click Workspace → Enter as any role
└─ View all projects within workspace
└─ View all members and their activity
└─ Configure/Update API integrations
└─ View usage analytics & logs
└─ Manage workspace settings
```

7. API Integration Architecture

7.1 Third-Party Services

Service Type	Purpose	Examples
Voice AI	Text-to-speech generation	ElevenLabs, Azure TTS, Google Cloud TTS, Amazon Polly
Video AI	Avatar/video generation	HeyGen, Synthesia, D-ID, Runway
AI Editing	Automated video editing	Subagic (specified)

Table 2 – continued

Service Type	Purpose	Examples
Publishing	Social media distribution	Zernio (specified), YouTube API, TikTok API, Meta API

7.2 Integration Pattern

```
[User Request] → [Next.js API Route] → [Workspace API Config Lookup] → [External API Call]
```

- └─ Encrypted API keys stored in workspace_apis table
- └─ Keys decrypted server-side only (never exposed to client)
- └─ Response cached per workspace for rate limiting

7.3 Webhook Handling

```
// API Route for voice generation completion
// app/api/webhooks/voice/route.ts

export async function POST(req: Request) {
  const payload = await req.json();
  const signature = req.headers.get('x-webhook-signature');

  // Verify webhook signature
  if (!verifyWebhookSignature(payload, signature)) {
    return new Response('Invalid signature', { status: 401 });
  }

  // Update voice generation record
  await supabase
    .from('voice_generations')
    .update({
      status: 'completed',
      audio_url: payload.audio_url,
      duration: payload.duration
    })
    .eq('id', payload.generation_id);

  // Trigger next stage (video generation) if auto-advance enabled
  await triggerNextStage(payload.project_id, 'voice');

  // Notify user
  await createNotification({
    user_id: payload.user_id,
    type: 'voice_complete',
    title: 'Voice generation complete',
    message: `Your voiceover for "${payload.project_title}" is ready.`
  });

  return new Response('OK', { status: 200 });
}
```

```
}
```

8. Authentication Flow

Phase 1: Password-Based (MVP)

1. Owner creates workspace
2. Owner invites members via email
3. System generates temp password + invite link
4. Member clicks link → Sets permanent password
5. Member can change password in Settings

Phase 2: Supabase Auth Integration (Post-MVP)

- OAuth providers (Google, GitHub, etc.)
- Magic Link authentication
- MFA support
- SSO for enterprise workspaces

Password Management Flow

```
Settings → Security
|
├─ Change Password
|   └─ Enter current password
|   └─ Enter new password + confirm
|   └─ Save → Update auth.users
|
└─ Reset Temp Password (first login)
    └─ Force password change on first login
    └─ Cannot access dashboard until changed
```

9. Notification System

9.1 Notification Types

Type	Trigger	Recipients
deadline	Admin creates scheduled task	Assigned user/editor
deadline_reminder	24h before deadline	Assigned user
edit_request	User sends video for editing	Editor
edit_complete	Editor finishes editing	Requesting user
edit_rejected	Editor declines task	Requesting user
publish_complete	Video published successfully	Project owner
publish_failed	Publishing error	Project owner + Admin
stage_complete	Workflow stage finished	Project owner
mention	User mentioned in comment	Mentioned user

Table 3 – continued

Type	Trigger	Recipients
invite	New member invited	Invited user
system	Platform announcements	All workspace members

9.2 Real-time Delivery

```
// Using Supabase Realtime for live notifications
const channel = supabase
  .channel('notifications')
  .on(
    'postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'notifications', filter: `user_id=eq.${userId}` },
    (payload) => {
      // Show toast notification
      toast.info(payload.new.title, { description: payload.new.message });
      // Update notification bell badge
      incrementUnreadCount();
    }
  )
  .subscribe();
```

10. Multi-Tenancy Security

10.1 Data Isolation Strategy: Shared Database, Shared Schema

- **Row-Level Security (RLS)** enforced on all tables via `workspace_id`
- Every query automatically scoped to user's workspace membership
- `tenant_id` (`workspace_id`) required on every row
- Composite indexes on (`workspace_id`, `id`) for performance

10.2 Key Security Measures

```
-- Ensure every table has workspace_id
-- All foreign keys include workspace_id for tenant scoping
-- RLS policies prevent cross-tenant reads

-- Example: Prevent direct table access without workspace filter
CREATE POLICY enforce_tenant_isolation ON projects
  FOR ALL
  USING (workspace_id IN (
    SELECT workspace_id FROM workspace_members WHERE user_id = auth.uid()
  ));
```

10.3 API Key Security

- API keys encrypted at rest using AES-256
- Decryption happens server-side only in API routes

- Keys never exposed to frontend
 - Per-workspace API rate limiting
 - Audit log for all API usage
-

11. Development Phases

Phase 1: Foundation (Weeks 1-4)

- Project scaffolding (Next.js + Supabase + Tailwind + shadcn/ui)
- Database schema implementation with RLS
- Basic auth (email/password via Supabase)
- Landing page + Auth pages
- Workspace CRUD (Owner)
- Member invitation system

Phase 2: Core Workflow (Weeks 5-8)

- Project creation & timeline UI
- Script generation interface
- Voice generation integration (mock API first)
- Video generation integration (mock API first)
- Basic project dashboard for Users

Phase 3: Editing & Publishing (Weeks 9-12)

- Editing task system (User → Editor)
- AI editing integration (Subagic)
- Manual editing workflow
- Publishing interface
- Zernio API integration
- Upload custom video + thumbnail

Phase 4: Admin Features (Weeks 13-14)

- Calendar / Scheduling system
- Team management (Admin)
- Notification system
- Activity logs & audit trail
- Owner workspace inspection

Phase 5: Polish & Scale (Weeks 15-16)

- Real-time updates (Supabase Realtime)
 - Advanced analytics
 - Performance optimization
 - Security hardening
 - Production deployment
-

12. Environment Variables

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Encryption (for API keys at rest)
ENCRYPTION_KEY=

# Third-Party API Keys (Platform-level defaults)
VOICE_API_KEY=
VIDEO_API_KEY=
SUBAGIC_API_KEY=
ZERNIO_API_KEY=

# App Config
NEXT_PUBLIC_APP_URL=http://localhost:3000
APP_ENV=development
```

13. Recommended Packages

```
{
  "dependencies": {
    "next": "^15.x",
    "react": "^19.x",
    "react-dom": "^19.x",
    "@supabase/supabase-js": "^2.x",
    "@supabase/auth-helpers-nextjs": "^0.x",
    "tailwindcss": "^4.x",
    "@radix-ui/react-*": "latest",
    "class-variance-authority": "latest",
    "clsx": "latest",
    "tailwind-merge": "latest",
    "lucide-react": "latest",
    "date-fns": "^3.x",
    "react-hook-form": "^7.x",
    "zod": "^3.x",
    "@hookform/resolvers": "latest",
    "react-query": "^5.x",
    "zustand": "^5.x",
    "recharts": "^2.x",
    "react-calendar": "^5.x",
    "uuid": "^11.x",
    "crypto-js": "^4.x"
  }
}
```

Document Version: 1.0 Last Updated: June 2026 Author: Platform Architecture Team